Tejaswi Khode BEB221

Experiment 9

Aim

Implement Text Similarity Recognizer for the chosen text documents.

Theory

Text similarity measures how alike two pieces of text are in terms of their content or meaning. It is widely used in NLP applications such as plagiarism detection, document clustering, information retrieval, and recommendation systems.

Common approaches to measure text similarity:

1. **Preprocessing** – Clean and normalize the text (lowercasing, removing punctuation/stopwords, tokenization).
2. **Vectorization** – Convert text into numerical form using techniques like Bag of Words or TF-IDF (Term Frequency–Inverse Document Frequency).
3. **Similarity Measure** – Compute similarity between the vector representations, most commonly using **Cosine Similarity**, which measures the cosine of the angle between two vectors (value ranges from 0 to 1, where 1 means identical).

Cosine Similarity Formula: `cos(θ) = (A · B) / (||A|| ||B||)`

Application

- Plagiarism detection
- Document clustering and classification
- Search engines / information retrieval
- Resume-to-Job Description matching

Advantages

- Simple and computationally efficient
- Works well with TF-IDF for capturing important terms
- Independent of document length (cosine similarity normalizes for magnitude)

Limitations

- Does not capture semantic meaning (synonyms are treated as different words)
- Sensitive to preprocessing quality
- Cannot capture word order or context (bag-of-words assumption)

In [1]:
```
import pandas as pd
import nltk
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /Users/peeyush/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     /Users/peeyush/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[1]: True

In [4]:
```
df = pd.read_csv("UpdatedResumeDataSet.csv")
print("Shape :", df.shape)
df.head()
```

```
Shape : (962, 2)
```

Out[4]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â€¢ R â€¢ Python â€¢ SAP HANA â€¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [5]:
```
print(df.columns)
df[['Category', 'Resume']].head()
```

```
Index(['Category', 'Resume'], dtype='str')
```

Out[5]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â€¢ R â€¢ Python â€¢ SAP HANA â€¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [6]:
```
doc1 = df['Resume'].iloc[0]
doc2 = df['Resume'].iloc[1]

print("Document 1 Category:", df['Category'].iloc[0])
print("Document 2 Category:", df['Category'].iloc[1])
print("\nDocument 1 (first 200 chars):\n", doc1[:200])
print("\nDocument 2 (first 200 chars):\n", doc2[:200])
```

```
Document 1 Category: Data Science
Document 2 Category: Data Science

Document 1 (first 200 chars):
 Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaScript/JQuery. * Machine lea
rning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision T

Document 2 (first 200 chars):
 Education Details
May 2013 to May 2017 B.E   UIT-RGPV
Data Scientist

Data Scientist - Matelabs
Skill Details
Python- Exprience - Less than 1 year months
Statsmodels- Exprience - 12 months
```

In [7]:
```
doc1 = df['Resume'].iloc[0]
doc2 = df['Resume'].iloc[1]

print("Document 1 Category:", df['Category'].iloc[0])
print("Document 2 Category:", df['Category'].iloc[1])
print("\nDocument 1 (first 200 chars):\n", doc1[:200])
print("\nDocument 2 (first 200 chars):\n", doc2[:200])
```

```
Document 1 Category: Data Science
Document 2 Category: Data Science

Document 1 (first 200 chars):
 Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaScript/JQuery. * Machine lea
rning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision T

Document 2 (first 200 chars):
 Education Details
May 2013 to May 2017 B.E   UIT-RGPV
Data Scientist

Data Scientist - Matelabs
Skill Details
Python- Exprience - Less than 1 year months
Statsmodels- Exprience - 12 months
```

In [8]:
```
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text)
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    return " ".join(tokens)

clean_doc1 = preprocess(doc1)
clean_doc2 = preprocess(doc2)

print("Cleaned Document 1:\n", clean_doc1[:300])
print("\nCleaned Document 2:\n", clean_doc2[:300])
```

```
Cleaned Document 1:
 skills programming languages python pandas numpy scipy scikit learn matplotlib sql java javascript jquery machine learning regressio
n svm na bayes knn random forest decision trees boosting techniques cluster analysis word embedding sentiment analysis natural langua
ge processing dimensionality reduct

Cleaned Document 2:
 education details may may b e uit rgpv data scientist data scientist matelabs skill details python exprience less year months statsm
odels exprience months aws exprience less year months machine learning exprience less year months sklearn exprience less year months
scipy exprience less year months ke
```

In [9]:
```
documents = [clean_doc1, clean_doc2]

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

print("TF-IDF Matrix Shape:", tfidf_matrix.shape)
print("\nFeature Names (sample):", vectorizer.get_feature_names_out()[:20])
```

```
TF-IDF Matrix Shape: (2, 382)

Feature Names (sample): ['accelerating' 'accounting' 'achievements' 'across' 'action' 'addresses'
 'aditya' 'advanced' 'also' 'analysis' 'analytic' 'analytics' 'analyzing'
 'angular' 'anomalies' 'answer' 'arima' 'art' 'asks' 'assisted']
```

In [10]:
```python
similarity_matrix = cosine_similarity(tfidf_matrix)
print("Similarity Matrix:\n", similarity_matrix)

similarity_score = similarity_matrix[0][1]
print(f"\nCosine Similarity between Document 1 and Document 2: {similarity_score:.4f}")
```

```
Similarity Matrix:
 [[1.         0.10931278]
 [0.10931278 1.        ]]

Cosine Similarity between Document 1 and Document 2: 0.1093
```

In [11]:
```python
# Two resumes from the SAME category
same_cat_df = df[df['Category'] == df['Category'].iloc[0]].reset_index(drop=True)
doc_a = preprocess(same_cat_df['Resume'].iloc[0])
doc_b = preprocess(same_cat_df['Resume'].iloc[1])

tfidf_same = vectorizer.fit_transform([doc_a, doc_b])
sim_same = cosine_similarity(tfidf_same)[0][1]

print("Category:", same_cat_df['Category'].iloc[0])
print(f"Similarity within SAME category: {sim_same:.4f}")

# Two resumes from DIFFERENT categories
diff_cat_df = df[df['Category'] != df['Category'].iloc[0]].reset_index(drop=True)
doc_c = preprocess(df['Resume'].iloc[0])
doc_d = preprocess(diff_cat_df['Resume'].iloc[0])

tfidf_diff = vectorizer.fit_transform([doc_c, doc_d])
sim_diff = cosine_similarity(tfidf_diff)[0][1]

print("\nCategories:", df['Category'].iloc[0], "vs", diff_cat_df['Category'].iloc[0])
print(f"Similarity across DIFFERENT categories: {sim_diff:.4f}")
```

```
Category: Data Science
Similarity within SAME category: 0.1093

Categories: Data Science vs HR
Similarity across DIFFERENT categories: 0.0350
```

In [12]:
```python
text1 = "Experienced Python developer with machine learning and data analysis skills."
text2 = "Skilled Python programmer specializing in machine learning and data science."

clean_text1 = preprocess(text1)
clean_text2 = preprocess(text2)

tfidf_custom = vectorizer.fit_transform([clean_text1, clean_text2])
custom_similarity = cosine_similarity(tfidf_custom)[0][1]

print("Text 1:", text1)
print("Text 2:", text2)
print(f"\nCosine Similarity: {custom_similarity:.4f}")
```

```
Text 1: Experienced Python developer with machine learning and data analysis skills.
Text 2: Skilled Python programmer specializing in machine learning and data science.

Cosine Similarity: 0.3361
```

Conclusion